
Pipeline di Riconoscimento — QualicodeService

Panoramica

Il software QualicodeService implementa un sistema di riconoscimento automatico di codici visivi apposti su targhe fisiche (“plate”). Le immagini vengono acquisite da telecamere IP e processate attraverso un pipeline multi-fase che comprende preprocessing geometrico, estrazione di feature, template matching e decodifica con correzione d’errore. Il sistema è progettato per operare in tempo reale, sfruttando elaborazione parallela su più stazioni di acquisizione.

Architettura del sistema

Il sistema è organizzato in tre livelli principali:

1. **Acquisition Layer** — Telecamere IP (protocolli HTTP/MJPEG e RTSP) inviano frame video al sistema. L’acquisizione può essere attivata da motion detection o su richiesta esplicita.
2. **Processing Layer** — Componenti C# gestiscono l’orchestrazione del flusso: smistamento dei frame ai thread di elaborazione, post-processing dei risultati, tracking e persistenza.
3. **Recognition Engine** — Una libreria nativa C++ (Qualicode.Lib.dll) esegue il riconoscimento vero e proprio, sfruttando OpenCV 4.7 per le operazioni di computer vision.

Pipeline di elaborazione

Fase 1 — Acquisizione e selezione ROI

Il frame acquisito dalla telecamera (tipicamente JPEG, risoluzione fino a 1920×1080) viene recapitato al componente Fetcher. Prima di essere passato al motore di riconoscimento, si applica una selezione della **Region of Interest (ROI)**: una sottoregione dell’immagine, definita per ogni stazione tramite quattro coordinate normalizzate (x, y, larghezza, altezza), che circoscrive la zona dove si trova fisicamente la targa.

Se la telecamera non è ortogonale alla targa, viene applicata una **correzione prospettica** (perspective correction) definita dai 4 vertici del quadrilatero sorgente (top-left, top-right, bottom-right, bottom-left), ciascuno con coordinate x e y normalizzate rispetto alle dimensioni dell’immagine. Questi 4 punti vengono usati per calcolare una trasformazione omografica che riproietta la ROI su un piano frontale virtuale rettangolare, eliminando la distorsione angolare.

Fase 2 — Trasformazioni geometriche

Prima del preprocessing, sull’immagine ritagliata vengono applicate le seguenti trasformazioni configurabili per stazione:

- **Ridimensionamento** — L’immagine viene scalata secondo i fattori `resizeWidth` e `resizeHeight` per normalizzare la risoluzione rispetto al codebook.
- **Rotazione immagine** — Rotazione planare in gradi (antioraria positiva) per compensare inclinazioni della telecamera.
- **Rotazione targa** — Rotazione in passi di 90° (0°, 90°, 180°, 270°) per gestire orientamenti fisici diversi della targa.
- **Flip** — Ribaltamento orizzontale o verticale per casi di montaggio speculare.

Fase 3 — Preprocessing dell'immagine

All'interno della libreria nativa viene eseguito il preprocessing dell'immagine tramite OpenCV:

- **Conversione del formato colore** — Il frame viene convertito dallo spazio colore originale al formato richiesto dall'algoritmo.
- **Normalizzazione dell'intensità** — Uniformazione del contrasto per ridurre le variazioni dovute all'illuminazione ambientale.
- **Filtraggio** — Applicazione di filtri per attenuare il rumore e migliorare la nitidezza dei bordi dei simboli.

Fase 4 — Localizzazione e riconoscimento dei simboli

Il cuore del riconoscimento avviene nella libreria nativa:

1. **Localizzazione simboli** — Attraverso tecniche di analisi dei contorni (contour detection) e blob analysis, il sistema individua le posizioni dei singoli simboli sulla targa, restituendo le coordinate del centroide di ciascuno.
2. **Estrazione delle feature** — Per ogni simbolo localizzato vengono estratte le feature visive (descrittori di forma) che ne caratterizzano l'aspetto.
3. **Template matching** — Le feature estratte vengono confrontate con un codebook preaddestrato. Il sistema supporta due modalità di ricerca, selezionabili via configurazione:
 - **Full search** — Ricerca completa, più accurata ma con latenza maggiore.
 - **Fast search** — Ricerca veloce, con minore accuratezza ma latenza ridotta.È possibile configurare la strategia di ricerca (solo full, solo fast, fast-then-full, ecc.) in base ai requisiti di performance.
4. **Decodifica Hamming** — Il risultato del matching produce una codeword grezza. Il codice adottato è un codice a correzione d'errore con parametri configurabili (numero di bit per simbolo n , distanza minima di Hamming d , numero di codeword possibili M). La decodifica produce l'indice della codeword riconosciuta e un punteggio di **affidabilità** (reliability, compreso tra 0 e 1).

Fase 5 — Decision logic

Il risultato della decodifica viene valutato rispetto a soglie di affidabilità minima configurabili:

- Se l'affidabilità è superiore alla soglia (default: 0.9 per full search, 0.8 per fast search) il riconoscimento è considerato **Success**.
- In caso di errori parziali entro limiti tollerati il risultato è **PartialSuccess**.
- Al di sotto delle soglie il frame viene scartato (**Fail**).

Fase 6 — Post-processing e tracking

Dopo il riconoscimento viene eseguito un post-processing in C#:

- **Plate cleaning (voting)** — Un meccanismo di accumulazione degli errori per simbolo su finestra temporale: se un determinato simbolo supera la soglia di errori, il sistema applica una correzione basata sul valore più frequente osservato.
- **Tracking posizione** — Viene estratto il rettangolo della targa (plateRect, 4 angoli) e calcolato il centroide. Con successive acquisizioni si determina la **direzione di movimento** della targa (statica, sinistra-destra, destra-sinistra, su-giù, giù-su) con un threshold di movimento configurabile (default: 64 pixel).
- **Sequencing** — Il sistema accumula riconoscimenti consecutivi: un evento di riconoscimento confermato viene inviato solo dopo un minimo di riconoscimenti coerenti in sequenza (default: 2 per Success, 3 per PartialSuccess), riducendo i falsi positivi.

Fase 7 — Output e persistenza

Una volta confermato il riconoscimento:

- Lo **snapshot** del frame viene salvato su disco (formato JPEG, qualità configurabile) in cartelle distinte per esito (Success, PartialSuccess, None, Fail, Error).
- L'**evento di riconoscimento** (con ID codeword, errori, affidabilità, direzione) viene pubblicato su un message broker RabbitMQ per l'integrazione con sistemi esterni.
- I dati vengono registrati nel **database SQLite** tramite NHibernate ORM.

Schema del flusso

